

Wandus Law RAG — Block-wise code walkthrough

Generated from the chatbot-pintu-demo repository for presentations. Each block maps to one slide or section.

Repository root: C:\Users\karth\Downloads\chatbot-pintu-demo

Block 1 — App startup (lifespan, Langfuse, OpenSearch, pricing loop)

Runs once at process start and cleans up on shutdown.

backend/app/main.py (lines 20–57)

```
20 @asynccontextmanager 21 async def lifespan(app: FastAPI): 22 from app.services.langfuse_tracing import
init_langfuse_client_from_settings 23 24 init_langfuse_client_from_settings() 25 settings = get_settings() 26 if
(settings.opensearch_url or "").strip(): 27 try: 28 ok = await search_index.ensure_index_exists() 29
logger.info("opensearch_index_ready=%s", ok) 30 except httpx.HTTPError as e: 31 logger.warning( 32 "OpenSearch
unreachable at startup (index not ensured): %s", 33 e, 34 ) 35 pricing_task: asyncio.Task | None = None 36 try: 37
from app.monitoring import llm_pricing 38 39 await llm_pricing.refresh_from_langfuse() 40 pricing_task =
asyncio.create_task(llm_pricing.pricing_refresh_loop()) 41 except Exception as e: 42 logger.debug("llm_pricing startup
skipped: %s", e) 43 44 yield 45 46 if pricing_task is not None: 47 pricing_task.cancel() 48 try: 49 await pricing_task
50 except asyncio.CancelledError: 51 pass 52 try: 53 from app.services.langfuse_tracing import flush_langfuse 54 55
flush_langfuse() 56 except Exception: 57 logger.debug("langfuse shutdown flush skipped", exc_info=True)
```

Block 2 — FastAPI app: middleware, /health, /ready, all routers

Single factory registers CORS, request IDs, and every feature router.

backend/app/main.py (lines 60–111)

```
60 class RequestIdMiddleware(BaseHTTPMiddleware): 61 async def dispatch(self, request: Request, call_next): 62 rid =
request.headers.get("x-request-id") or str(uuid.uuid4()) 63 request.state.request_id = rid 64 response = await
call_next(request) 65 response.headers["X-Request-ID"] = rid 66 return response 67 68 69 def create_app() -> FastAPI:
70 get_settings() 71 app = FastAPI( 72 title="Wandus Law RAG API", 73 version="0.1.0", 74 lifespan=lifespan, 75 ) 76
app.add_middleware(RequestIdMiddleware) 77 app.add_middleware( 78 CORSMiddleware, 79 allow_origins=["*"], 80
allow_credentials=True, 81 allow_methods=["*"], 82 allow_headers=["*"], 83 ) 84 85 @app.get("/health") 86 async def
health(): 87 return {"status": "ok"} 88 89 @app.get("/ready") 90 async def ready(): 91 ok = await check_db_ready() 92
if not ok: 93 raise HTTPException(status_code=503, detail="not_ready") 94 return {"status": "ready"} 95 96
app.include_router(ingest.router) 97 app.include_router(documents.router) 98 app.include_router(agent_graph.router) 99
app.include_router(chat.router) 100 app.include_router(memory.router) 101 app.include_router(users.router) 102
app.include_router(user_chat_state.router) 103 app.include_router(eval_metrics.router) 104
app.include_router(titles.router) 105 app.include_router(usage_dashboard.router) 106
app.include_router(monitoring.router, prefix="/monitoring") 107 app.include_router(monitoring.router,
prefix="/api/v1/monitoring") 108 # Jam with AI course-compatible API ( /ask, /stream, /hybrid-search ) 109
app.include_router(jam_api.router, prefix="/api/v1") 110 111 return app
```

Block 3 — Settings from .env (Pydantic BaseSettings)

Typed configuration: DATABASE_URL, OpenAI keys, models, memory flags.

backend/app/config.py (lines 11–43)

```
11 class Settings(BaseSettings): 12 model_config = SettingsConfigDict( 13 env_file=".env", 14
env_file_encoding="utf-8", 15 extra="ignore", 16 ) 17 18 database_url: str = Field(validation_alias="DATABASE_URL") 19
openai_api_key: str = Field(validation_alias="OPENAI_API_KEY") 20 21 openai_embedding_model: str = Field( 22
default="text-embedding-3-small", 23 validation_alias="OPENAI_EMBEDDING_MODEL", 24 ) 25 openai_chat_model: str =
Field( 26 default="gpt-4o-mini", 27 validation_alias="OPENAI_CHAT_MODEL", 28 ) 29 embedding_dimensions: int = Field(
30 default=1536, 31 validation_alias="EMBEDDING_DIMENSIONS", 32 ) 33 34 # Long-term memory vectors: size is
`app.embedding_dims.MEMORY_ITEM_EMBED_DIM` (384); 35 # use OpenAI `text-embedding-3-*` with reduced dimensions, or
free local Sentence-Transformers. 36 memory_embedding_provider: MemoryEmbeddingProvider = Field( 37 default="openai",
38 validation_alias="MEMORY_EMBEDDING_PROVIDER", 39 ) 40 sentence_transformer_memory_model: str = Field( 41
default="sentence-transformers/all-MiniLM-L6-v2", 42 validation_alias="SENTENCE_TRANSFORMER_MEMORY_MODEL", 43 )
```

Block 4 — Async SQLAlchemy engine and get_session dependency

Converts postgresql:// to asyncpg URL; session per HTTP request.

backend/app/database.py (lines 12–41)

```
12 def _async_database_url(url: str) -> str: 13 if url.startswith("postgresql+asyncpg://"): 14 return url 15 if
url.startswith("postgresql://"): 16 return "postgresql+asyncpg://" + url[len("postgresql://") :] 17 return url 18 19
20 def get_engine(): 21 global _engine 22 if _engine is None: 23 settings = get_settings() 24 _engine =
create_async_engine( 25 _async_database_url(settings.database_url), 26 echo=False, 27 poolclass=NullPool, 28 ) 29
return _engine 30 31 32 def get_session_factory(): 33 global _session_factory 34 if _session_factory is None: 35
_session_factory = async_sessionmaker( 36 get_engine(), 37 class_=AsyncSession, 38 expire_on_commit=False, 39
autoflush=False, 40 ) 41 return _session_factory
```

Block 4b — get_session generator

backend/app/database.py (lines 53–56)

```
53 async def get_session() -> AsyncGenerator[AsyncSession, None]: 54 factory = get_session_factory() 55 async with
factory() as session: 56 yield session
```

Block 5 — Chat router: JSON POST /chat

prepare_retrieval_scope → chat_dispatch → optional auto_memory → response.

backend/app/routers/chat.py (lines 21–53)

```
21 @router.post("", response_model=ChatResponse) 22 async def chat( 23 body: ChatBody, 24 session: AsyncSession =
Depends(get_session), 25 ): 26 hist = [h.model_dump() for h in body.history] if body.history else None 27 scope_ids =
await prepare_retrieval_scope( 28 session, 29 body.question, 30 hist, 31 body.document_id, 32 ) 33 lf_sid =
str(uuid.uuid4()) 34 answer, citations, source = await chat_dispatch( 35 session, 36 question=body.question, 37
top_k=body.top_k, 38 history=hist, 39 scope_document_ids=scope_ids, 40 langfuse_session_id=lf_sid, 41
memory_user_id=body.memory_user_id, 42 ) 43 if body.auto_memory and get_settings().auto_memory_from_conversation and
answer.strip(): 44 try: 45 n = await persist_auto_memory_from_turn( 46 session, body.memory_user_id, body.question,
answer 47 ) 48 if n: 49 await session.commit() 50 except Exception: 51 logger.exception("auto_memory after non-stream
chat failed") 52 53 return ChatResponse(answer=answer, citations=citations, source=source)
```

Block 5b — Streaming POST /chat/stream (SSE)

Yields data: {json}\n\n events for tokens and metadata.

backend/app/routers/chat.py (lines 66–109)

```
66 async def event_stream(): 67 settings = get_settings() 68 do_auto_memory = ( 69 body.auto_memory 70 and
settings.auto_memory_from_conversation 71 ) 72 saw_error = False 73 token_parts: list[str] = [] 74 75 hist =
[h.model_dump() for h in body.history] if body.history else None 76 scope_ids = await prepare_retrieval_scope( 77
session, 78 body.question, 79 hist, 80 body.document_id, 81 ) 82 lf_sid = str(uuid.uuid4()) 83 async for ev in
stream_chat_dispatch( 84 session, 85 question=body.question, 86 top_k=body.top_k, 87 history=hist, 88
scope_document_ids=scope_ids, 89 langfuse_session_id=lf_sid, 90 memory_user_id=body.memory_user_id, 91 ): 92 if
ev.get("event") == "error": 93 saw_error = True 94 if ev.get("event") == "token" and ev.get("text"): 95
token_parts.append(ev["text"]) 96 yield f"data: {json.dumps(ev)}\n\n" 97 98 answer = "".join(token_parts) 99 if
do_auto_memory and not saw_error and answer.strip(): 100 try: 101 n = await persist_auto_memory_from_turn( 102
session, body.memory_user_id, body.question, answer 103 ) 104 if n: 105 await session.commit() 106 except Exception:
107 logger.exception("auto_memory after stream failed") 108 109 return StreamingResponse(event_stream(),
media_type="text/event-stream")
```

Block 6 — RAG: memory snippets, chunk count, vector retrieval

embed_texts + cosine_distance on Chunk.embedding; optional document scope.

backend/app/services/rag.py (lines 40–85)

```
40 async def fetch_memory_snippets( 41 session: AsyncSession, 42 memory_user_id: str | None, 43 question: str, 44 ) ->
list[str]: 45 uk = normalize_user_key(memory_user_id) 46 if not uk: 47 return [] 48 try: 49 return await
retrieve_memory_texts(session, uk, question) 50 except Exception as e: 51 # e.g. UndefinedTableError when ``alembic
upgrade head`` has not been run 52 logger.warning("long-term memory retrieval skipped (chat continues): %s", e) 53
return [] 54 55 56 async def count_chunks(session: AsyncSession) -> int: 57 n = await
session.scalar(select(func.count()).select_from(Chunk)) 58 return int(n or 0) 59 60 61 async def retrieve_chunks( 62
session: AsyncSession, 63 question: str, 64 top_k: int, 65 *, 66 scope_document_ids: list[UUID] | None = None, 67 ) ->
list[tuple[Chunk, Document, float]]: 68 """Returns (chunk, document, distance) ordered by cosine distance
ascending.""" 69 (qvec,) = await embed_texts([question]) 70 dist_col = Chunk.embedding.cosine_distance(qvec) 71 72
stmt = ( 73 select(Chunk, Document, dist_col.label("dist")) 74 .join(Document, Chunk.document_id == Document.id) 75
.order_by(dist_col) 76 .limit(top_k) 77 ) 78 if scope_document_ids: 79 stmt =
stmt.where(Chunk.document_id.in_(scope_document_ids)) 80 81 rows = (await session.execute(stmt)).all() 82 out:
list[tuple[Chunk, Document, float]] = [] 83 for chunk, doc, dist in rows: 84 out.append((chunk, doc, float(dist))) 85
return out
```

Block 7 — Ingest router: JSON and PDF multipart

Delegates to ingest_document service after PDF text extraction.

backend/app/routers/ingest.py (lines 12–55)

```
12 @router.post("", response_model=IngestResponse) 13 async def ingest_json( 14 body: IngestBody, 15 session:
AsyncSession = Depends(get_session), 16 ): 17 doc_id, n = await ingest_document( 18 session, 19 title=body.title, 20
text=body.text, 21 source_uri=body.source_uri, 22 ) 23 return IngestResponse(document_id=doc_id, chunks_created=n) 24
25 26 @router.post("/file", response_model=IngestResponse) 27 async def ingest_file( 28 title: str = Form(...,
min_length=1, max_length=512), 29 source_uri: str | None = Form(None), 30 file: UploadFile = File(...), 31 session:
AsyncSession = Depends(get_session), 32 ): 33 name = (file.filename or "").strip().lower() 34 if not
name.endswith(".pdf"): 35 raise HTTPException( 36 status_code=400, 37 detail="Only PDF uploads are supported. Use a
.pdf file.", 38 ) 39 raw = await file.read() 40 try: 41 text = bytes_to_document_text(file.filename, raw) 42 except
Exception as e: 43 raise HTTPException( 44 status_code=400, 45 detail=f"Could not read file: {e!s}", 46 ) from e 47 if
not text.strip(): 48 raise HTTPException( 49 status_code=400, 50 detail="No extractable text in file (empty PDF or
unsupported content).", 51 ) 52 doc_id, n = await ingest_document( 53 session, title=title, text=text,
source_uri=source_uri 54 ) 55 return IngestResponse(document_id=doc_id, chunks_created=n)
```

Block 8 — Vite dev server: /api proxy to FastAPI

Browser uses /api/...; Vite strips prefix and forwards to port 8000.

frontend/vite.config.ts (lines 4–18)

```
4 export default defineConfig({ 5 plugins: [react()], 6 server: { 7 host: "127.0.0.1", 8 port: 5173, 9 strictPort:
true, 10 proxy: { 11 "/api": { 12 target: "http://127.0.0.1:8000", 13 changeOrigin: true, 14 rewrite: (path) =>
path.replace(/^\/api/, ""), 15 }, 16 }, 17 }, 18 });
```

Block 9 — Frontend SSE consumer (consumeChatSse)

Reads fetch stream, splits on blank line, parses data: JSON events.

frontend/src/App.tsx (lines 32–79)

```
32 type StreamEvent = 33 | { event: "graph_step"; node?: string } 34 | { event: "meta"; citations?: Citation[];
source?: string } 35 | { event: "token"; text?: string } 36 | { event: "done" } 37 | { event: "error"; detail?: string
}; 38 39 async function consumeChatSse( 40 body: object, 41 onEvent: (ev: StreamEvent) => void 42 ): Promise<void> {
43 const r = await fetch(`${API}/chat/stream`, { 44 method: "POST", 45 headers: { "Content-Type": "application/json"
}, 46 body: JSON.stringify(body), 47 }); 48 if (!r.ok) throw new Error(await r.text()); 49 const reader =
r.body?.getReader(); 50 if (!reader) throw new Error("No response body"); 51 const decoder = new TextDecoder(); 52 let
buf = ""; 53 while (true) { 54 const { done, value } = await reader.read(); 55 if (value) buf += decoder.decode(value,
```

```
{ stream: true }); 56 let sep: number; 57 while ((sep = buf.indexOf("\n\n")) >= 0) { 58 const block = buf.slice(0,
sep); 59 buf = buf.slice(sep + 2); 60 const line = block.trim(); 61 if (line.startsWith("data: ")) { 62 try { 63
onEvent(JSON.parse(line.slice(6)) as StreamEvent); 64 } catch { 65 /* ignore malformed chunk */ 66 } 67 } 68 } 69 if
(done) break; 70 } 71 const tail = buf.trim(); 72 if (tail.startsWith("data: ")) { 73 try { 74
onEvent(JSON.parse(tail.slice(6)) as StreamEvent); 75 } catch { 76 /* ignore */ 77 } 78 } 79 }
```

End of document. Re-run: `python generate_walkthrough_pdf.py`